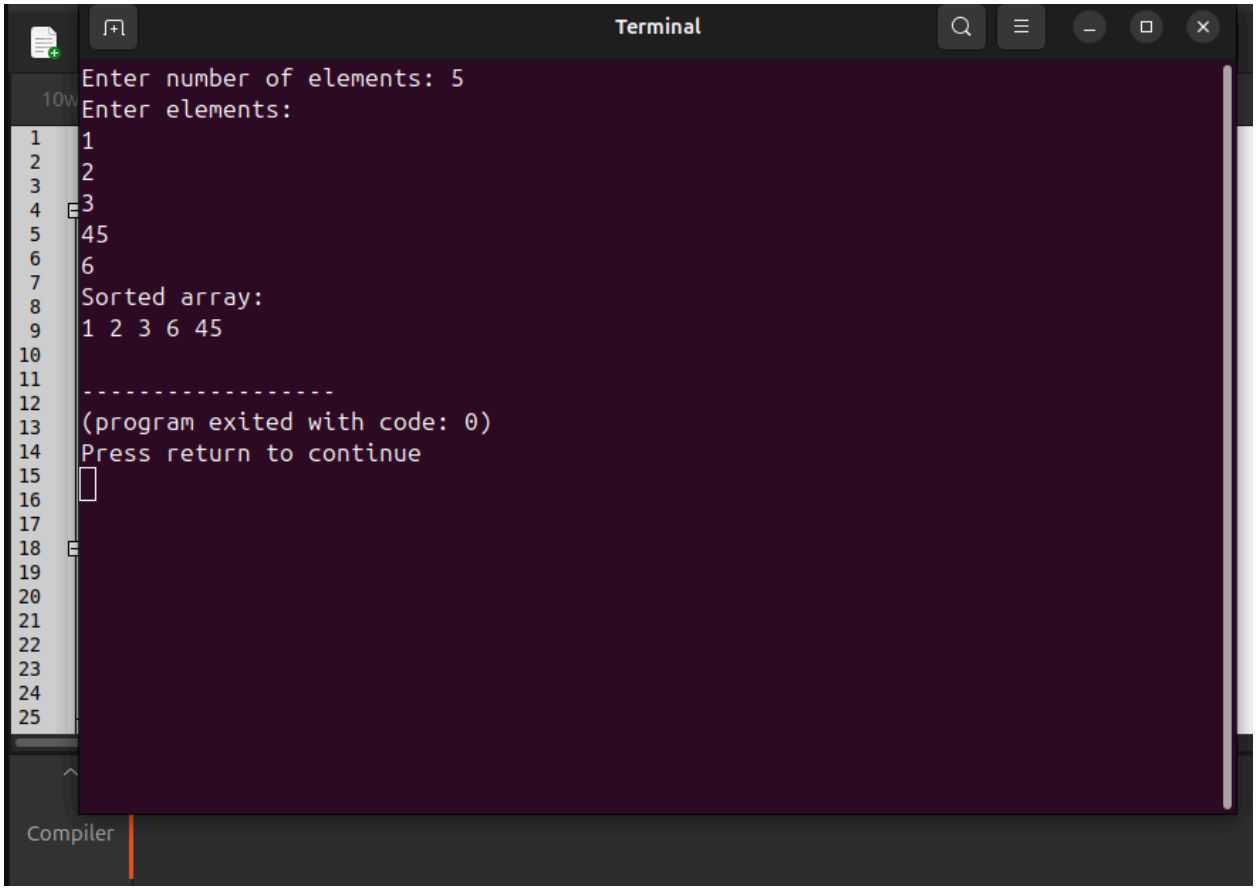
```c
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;        // Assume root is largest
    int left = 2 * i + 1;   // Left child
    int right = 2 * i + 2;  // Right child

    // If left child is larger
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify affected subtree
        heapify(arr, n, largest);
    }
}

// Heap Sort function
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        // Swap root with last element
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // Heapify reduced heap
        heapify(arr, i, 0);
    }
}
```

```
// Print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

// Driver code
int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array:\n");
    printArray(arr, n);

    return 0;
}
```


Terminal
Enter number of elements: 5
Enter elements:
1
2
3
45
6
Sorted array:
1 2 3 6 45

(program exited with code: 0)
Press return to continue
Compiler